MedStock: Pharmacy Inventory Analytics using Data Warehousing and Data Mining

Subject: Data Warehousing and Mining (DWM) Mini Project

Project topic: MedStock – a pharmacy data warehouse that mines sales patterns, forecasts demand and recommends stock actions.

Project Guide: Prof. Shruti Agrawal

Submitted by:

Sr.	Student name	Roll no.
1	Himanshu Choyal	24101B0035
2	Sujal Tiwari	24101B0028
3	Gayatri Chand	24101B0004

1. Introduction and objectives

Pharmacies lose money in two ways: medicines run out when customers need them, and stock expires on the shelf. MedStock is an end-to-end DWM project that attacks both problems with a data warehouse, data mining and demand forecasting.

Because real pharmacy data is private, the project generates realistic synthetic data: 500 medicines, 5 branches and 2 years of sales, purchases and batches. The same pipeline can also run on a user's own uploaded CSV files.

Objectives:

- Build a star-schema data warehouse in PostgreSQL using an ETL process.
- Answer business questions with SQL analytics and OLAP operations.
- Mine patterns: association rules, clusters and anomalies.
- Forecast demand for 7, 14 and 30 days.
- Turn results into clear actions (reorder, overstock, expiry) shown on a web dashboard.

2. Data and warehouse design

The warehouse follows a star schema: fact tables for sales, purchases and inventory sit in the centre, surrounded by dimensions for date, medicine, branch, supplier and category.

Data generation. Daily demand per medicine and branch is simulated by multiplying a base level by branch size, weekday, season, trend and random noise. Stock is replenished with a reorder-point rule, and sales use FEFO (first-expiry-first-out). Customer baskets include planted "bought together" pairs, and some spikes are planted as anomalies so the mining can be checked later.

ETL. Python scripts extract the CSV files, validate and clean them, assign surrogate keys and load PostgreSQL. Row counts and totals are checked against the source after loading.

OLAP and analytics. SQL queries use GROUP BY, ROLLUP and window functions (RANK, running share). They report revenue by branch, month and category, seasonal index, demand variability (coefficient of variation), days of cover and expiry risk. The dashboard also has an OLAP Explorer with slice, dice, roll-up, drill-down and pivot over time, location and product hierarchies.

3. Data mining techniques

Three mining tasks run on the warehouse data, plus a short forecasting step.

Technique	Purpose	Key setting	Result
Apriori association rules	Find medicines bought together	min support 0.05%, confidence 10%, lift > 1	14 rules over 8 medicine pairs, lift ≥ 21.46
K-Means clustering (with PCA for plotting)	Group medicines by behaviour	K chosen by silhouette score	K = 4 segments, silhouette 0.22; the 94-medicine high-volume segment carries 67.5% of units
Anomaly detection (robust z-score + Isolation Forest)	Spot unusual sales, stockouts and purchases	cut-off 5 (week), 6 (day)	8,159 rows flagged in 4,744 episodes; 72% of the 150 planted events were found
Gradient boosting forecast	Predict demand for 7, 14, 30 days	chronological train/validation/test split	Test MAE 2.40 (7d), 3.67 (14d), 6.09 (30d); lower error than every simple baseline at every horizon

All baselines use only past data, so there is no data leakage. Anomaly flags mean "unusual", not "proven wrong", and association rules show co-occurrence, not cause and effect.

Model evaluation matrix

1. Gradient Boosting — Demand Forecasting

Evaluation metric	**7-day**	**14-day**	**30-day**
MAE ↓	2.4030	3.6728	6.0913
RMSE ↓	4.2063	6.4622	10.9181
MASE ↓	0.7888	0.8338	0.9230
Bias	−11.37%	−8.60%	−6.57%

↓ *Lower is better.*

2. K-Means — Medicine Clustering

Evaluation metric	Result for K = 4
Number of clusters (K)	4
Inertia ↓	2527.6
Silhouette score ↑	0.2201
Calinski–Harabasz score ↑	161.73
Davies–Bouldin index ↓	1.3579

↑ *Higher is better.*

3. Anomaly Detection — Robust Z-score and Isolation Forest

The report evaluates the individual detectors and their combined results against 150 planted anomalies.

Detector	Precision ↑	Strict Recall ↑	F1-score ↑	Episodes flagged
Robust Z-score	0.0874	0.5133	0.1494	1,808
Isolation Forest	0.0798	0.6800	0.1428	3,659
Both methods combined	0.1990	0.4200	0.2700	578
Either method	0.0677	0.7200	0.1237	4,744

Higher precision, recall and F1 are generally preferred.

4. Apriori — Association Rule Mining

Evaluation metric	Result
Total association rules	14
Medicine pairs identified	8
Minimum support setting	0.05%
Minimum confidence setting	10%
Support across reported rules	0.07%–0.92%
Confidence across reported rules	0.15–0.81
Lift across reported rules	21.5–168.7

4. Decision support and system

The data flows: generator or user upload → validation → ETL → PostgreSQL warehouse → SQL analytics, mining and forecasting → decision rules → FastAPI → React dashboard.

Decision rules are explainable, not black-box. Days of cover (stock ÷ daily demand) sets stockout risk: under 3 days is CRITICAL, under 7 HIGH, under 14 MEDIUM. Safety stock is z × demand std × √lead time with z = 1.645 (95% service level). Overstock and near-expiry batches are flagged and merged into one prioritised action queue.

The dashboard has pages for Dashboard, Sales (with OLAP Explorer), Inventory, Risk, Insights, Forecast, Decisions and Upload Data.

5. Conclusion and limitations

MedStock shows the full DWM life cycle on one problem: data generation, ETL into a star-schema warehouse, OLAP analysis, mining and a dashboard that tells a pharmacy manager what to reorder, what is overstocked and what to sell before it expires.

Limitations: the demo data is synthetic, so results show how the methods behave rather than real pharmacy facts. For uploaded data, stock is reconstructed and therefore estimated. Lead time (7 days), service level (95%) and order cover (14 days) are assumptions.

Future work: FP-Growth for faster rules, other clustering methods (DBSCAN), a scheduled nightly pipeline, login roles and Docker packaging.